Assignment: Data Structures & Systems Design

Problem 1: LRU Cache Implementation

My Approach

To achieve strict O(1) time complexity for both get and put operations while maintaining recency order, two fundamental requirements must be satisfied:

1. Instant key-based lookup (O(1)).
2. Instant node removal and re-insertion at the most recently used (MRU) position (O(1)).

A hash map alone provides O(1) lookups but lacks ordering. An array maintains ordering, but moving an accessed item to the front requires shifting elements, costing O(n) time. A singly linked list allows O(1) insertions, but removing an intermediate node requires traversing from the head to find its predecessor, also taking O(n) time.

Therefore, I combined a hash map with a doubly linked list:

• The hash map maps each key directly to its corresponding doubly linked list Node object.

• The doubly linked list maintains access order (head = MRU, tail = LRU). Because each node holds direct references to its prev and next neighbors, unlinking and moving any node to the head is an O(1) pointer reassignment.

• Sentinel nodes (head and tail) act as permanent boundary markers, eliminating conditional checks for empty lists or boundary insertions.

Code

```
class Node:
    """A node in the doubly linked list,
holding one cached key-value pair."""
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None


class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = {}  # key -> Node

        # Sentinel nodes -- head.next is MRU,
tail.prev is LRU
        self.head = Node(0, 0)
        self.tail = Node(0, 0)
        self.head.next = self.tail
        self.tail.prev = self.head

    def _remove(self, node):
        """Detach a node from its current
position in the list."""
        prev_node = node.prev
        next_node = node.next
        prev_node.next = next_node
        next_node.prev = prev_node

    def _add_to_front(self, node):
        """Place a node right after head (most
recently used position)."""
        node.prev = self.head
        node.next = self.head.next
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.cache:
            return -1

        node = self.cache[key]
        # Move to front -- this item was just
accessed
        self._remove(node)
        self._add_to_front(node)
        return node.value

    def put(self, key, value):
        if key in self.cache:
            # Remove old node so we can re-
insert with the new value
            self._remove(self.cache[key])
            del self.cache[key]
```

```
        new_node = Node(key, value)
        self._add_to_front(new_node)
        self.cache[key] = new_node

        if len(self.cache) > self.capacity:
            # Evict the least recently used item
(just before tail)
            lru_node = self.tail.prev
            self._remove(lru_node)
            del self.cache[lru_node.key]
```

Verification

```
cache = LRUCache(2)
cache.put(1, 1)
cache.put(2, 2)
print(cache.get(1))    # 1  -- key 1 is now most recent
cache.put(3, 3)        # evicts key 2 (it's the LRU)
print(cache.get(2))    # -1 -- key 2 was evicted
cache.put(4, 4)        # evicts key 1
print(cache.get(1))    # -1 -- key 1 was evicted
print(cache.get(3))    # 3
print(cache.get(4))    # 4
```

Problem 2: Event Scheduler

My Approach -: can_attend_all

To determine whether a single person can attend all scheduled events without conflict:
1. We sort events by their start time in O(n log n) time.
2. We perform a single linear scan (O(n)), verifying whether any event begins before the preceding one concludes.
3. An overlap occurs if and only if current_event.start < previous_event.end.
Because adjacent meetings (e.g., (9, 10) and (10, 11)) do not overlap, we enforce a strict inequality (<), allowing back-to-back attendance without penalty.

My Approach -: min_rooms_required

To find the minimum number of rooms needed, we calculate the peak number of concurrent events across time:
1. Timeline Decomposition: Each interval (start, end) is split into two discrete boundary points: (start, +1) indicating a room being claimed, and (end, -1) indicating a room being freed.
2. Chronological Sorting & Tie-Breaking: We sort all points by timestamp. When an event end and another event start coincide (e.g., end at 10 and start at 10), the release (-1) is processed before the allocation (+1). This frees the room before assigning it, adhering to the non-overlapping adjacent event constraint.
3. Sweep-Line Scan: A single linear pass aggregates the running count of active rooms. The peak value observed during the sweep is the minimum room capacity required.

Code

```
def can_attend_all(events):
    """Check if one person can attend all events without
conflicts."""
    if len(events) <= 1:
        return True

    sorted_events = sorted(events, key=lambda e: e[0])

    for i in range(1, len(sorted_events)):
        if sorted_events[i][0] < sorted_events[i - 1][1]:
            return False

    return True


def min_rooms_required(events):
    """Find the minimum number of rooms needed for all
events."""
    if not events:
        return 0

    timeline = []
    for start, end in events:
        timeline.append((start, 1))    # meeting begins
        timeline.append((end, -1))     # meeting ends

    # Sort by time; at same time, process ends before
starts
    timeline.sort(key=lambda point: (point[0], point[1]))

    rooms_in_use = 0
    peak = 0

    for _, change in timeline:
        rooms_in_use += change
        if rooms_in_use > peak:
            peak = rooms_in_use

    return peak
```

Verification

```
# Adjacent events -- all fine, 1 room
print(can_attend_all([(9, 10), (10, 11), (11, 12)]))       # True
print(min_rooms_required([(9, 10), (10, 11), (11, 12)]))   # 1

# Overlapping events
print(can_attend_all([(9, 11), (10, 12)]))                 # False
print(min_rooms_required([(1, 4), (2, 5), (3, 6)]))        # 3
```

Final Discussion & Analysis

1. Time & Space Complexity

LRU Cache:

Operation	Time	Space
__init__(capacity)	O(1)	O(capacity)
get(key)	O(1)	O(1)
put(key, value)	O(1)	O(1) per call

Hash map lookups and updates execute in O(1) average time. Node unlinking and head insertions involve four pointer updates, executing in deterministic O(1) time. Auxiliary space is O(capacity) to store up to 'capacity' nodes and hash map entries.

Event Scheduler:

Function	Time	Space
can_attend_all(events)	O(n log n)	O(n)
min_rooms_required(events)	O(n log n)	O(n)

For both functions, the computational bottleneck is sorting in O(n log n) time using Timsort. Subsequent linear passes operate in O(n) time. Auxiliary space is O(n) to store the sorted copy and the 2n timeline points.

2. Trade-offs: Why HashMap + Doubly Linked List?

The combination of a hash map and a doubly linked list represents a classic time-space trade-off optimized for constant-time cache performance:

• Hash Map alone: Provides O(1) key lookup, but lacks recency ordering. Evicting the LRU element would require an O(n) linear scan across all entries.
• Array / Dynamic List: Maintains access order, but promoting an accessed element to the front requires shifting all intervening elements, incurring an O(n) cost.
• Singly Linked List: Deleting a node requires access to its predecessor. Without a backward pointer, locating the predecessor requires an O(n) traversal from the head.
• Doubly Linked List + Hash Map: Each node contains prev and next pointers, enabling O(1) unlinking. The hash map stores pointers directly to these nodes, bridging O(1) key lookups to O(1) list mutations.

Trade-off: We trade auxiliary memory (two pointers per node plus hash map overhead) to achieve guaranteed O(1) latency on all read and write paths. For hot-path caching layers, this overhead is well justified.

3. Future-Proofing: Assigning Specific Room Numbers

To extend the scheduler from counting total rooms to assigning concrete identifiers (e.g., 'Room A', 'Room B'):
1. Chronological Sorting: Sort all events by start time in O(n log n) time.

2. Min-Heap Priority Queue: Maintain a min-heap storing tuples of (end_time, room_identifier), ordered by the earliest release time.
3. Room Reuse Logic: For each incoming event, check the heap root. If root.end_time <= event.start, pop the room and return its identifier to an available room pool.
4. Assignment: Assign an identifier from the pool (reusing a freed room) or mint a new room if the pool is empty. Push (event.end, room_identifier) back into the heap.

This maintains optimal O(n log n) runtime and O(n) space while greedily minimizing the total rooms allocated.

```
import heapq

def assign_rooms(events):
    if not events:
        return []

    indexed = sorted(enumerate(events), key=lambda x: x[1][0])
    assignments = [None] * len(events)

    room_counter = 0
    available_rooms = []  # pool of freed room names
    occupied = []         # min-heap of (end_time, room_name)

    for orig_idx, (start, end) in indexed:
        # Free up rooms that have finished by now
        while occupied and occupied[0][0] <= start:
            _, freed_room = heapq.heappop(occupied)
            available_rooms.append(freed_room)

        # Pick a room
        if available_rooms:
            room = available_rooms.pop()
        else:
            room = f"Room {chr(65 + room_counter)}"
            room_counter += 1

        assignments[orig_idx] = room
        heapq.heappush(occupied, (end, room))

    return list(zip(events, assignments))
```

4. Concurrency: Making the LRU Cache Thread-Safe

In multi-threaded environments, concurrent invocations of get() and put() cause critical race conditions: corrupted doubly linked list pointers during unlinking, lost updates in the hash map, or cache size inconsistencies exceeding capacity.

1. Coarse-Grained Synchronization (Primary Approach):
Wrap all public methods (get and put) with a mutual exclusion lock (threading.Lock). This ensures mutual exclusion across both the hash map and linked list, guaranteeing safety with minimal complexity and negligible overhead under moderate workloads.

```
import threading

class ThreadSafeLRUCache(LRUCache):
    def __init__(self, capacity):
        super().__init__(capacity)
        self._lock = threading.Lock()

    def get(self, key):
        with self._lock:
            return super().get(key)

    def put(self, key, value):
        with self._lock:
            return super().put(key, value)
```

2. Read-Write Lock Considerations:
Although caches typically serve read-heavy workloads, get() in an LRU cache is fundamentally a mutating write operation because it alters the linked list to promote recency. Consequently, a standard RWLock does not improve read concurrency unless recency updates are decoupled or batched into a separate worker thread.

3. High-Throughput Alternatives:
For high-concurrency systems, lock contention can be mitigated by cache sharding (partitioning keys across N independent sub-caches via consistent hashing) or by using lock-free read buffers with CAS primitives.